

LAB 03

POINT PROCESSING

CÁC PHÉP BIẾN ĐỔI ĐIỂM TRÊN ẢNH SỐ

Giảng viên biên soạn: Võ Quang Hoàng Khang

1. Mục tiêu bài thực hành

- Giải thích được mô hình point processing $s = T(r)$, trong đó output tại (x,y) chỉ phụ thuộc pixel input cùng vị trí.
- Phân biệt point processing với xử lý lân cận và xử lý toàn cục.
- Kiểm soát kiểu dữ liệu ảnh 8-bit, overflow/underflow và clipping về $[0,255]$.
- Cài đặt bằng NumPy các phép tăng/giảm sáng, blending, negative, log, gamma và thresholding.
- Tách và tái tạo các mặt phẳng bit bằng toán tử bit.
- Tạo padding Constant, Replicate và Reflect; giải thích ảnh hưởng của padding đến kích thước và biên ảnh.
- Hiển thị ảnh, histogram và các thống kê min/max/mean để phân tích kết quả.
- Đối chứng kết quả NumPy với hàm OpenCV tương đương khi có, thay vì chỉ gọi thư viện.

2. Chuẩn bị và yêu cầu môi trường

- Tài khoản Google và Google Colab.
- Python 3.x; NumPy; OpenCV; Matplotlib.
- Ít nhất 2 ảnh grayscale 8-bit và 2 ảnh cùng kích thước để thử blending.
- Ôn biểu diễn ảnh số, miền $[0,255]$, ma trận ảnh và histogram.

Khởi tạo môi trường

```
# Cell 1 - Thiết lập môi trường dùng chung
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV:", cv2.__version__)
print("NumPy :", np.__version__)

# TODO: upload/đọc ảnh xám
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise ValueError("Không đọc được ảnh")

print("shape:", img.shape)
print("dtype:", img.dtype)
print("min/max/mean:", img.min(), img.max(), img.mean())
```

3. Lý thuyết tóm tắt và quy ước

Quy ước chung: xử lý ảnh 8-bit trong miền $[0,255]$; chuyển sang kiểu số thích hợp trước phép toán, clip và làm tròn theo quy định trước khi chuyển về uint8. Sinh viên phải đọc công thức, dự đoán xu hướng kết quả và giải thích bằng lời trước khi chạy code.

4. Phần A - Kiểu dữ liệu và các phép toán số học

Bài 1. Kiểm soát kiểu dữ liệu, overflow/underflow và clipping

Mục tiêu

Nhận biết lỗi khi thao tác số học trực tiếp trên uint8 và xây dựng pipeline an toàn.

Cơ sở lý thuyết

Miền ảnh 8-bit: $r \in \{0,1,\dots,255\}$.

$s = \text{clip}(r+c, 0, 255)$, với $\text{clip}(x,a,b) = \min(\max(x,a),b)$.

Nếu tính trên số nguyên không dấu 8-bit theo modulo: $s_wrap = (r+c) \bmod 256$. Đây không phải clipping.

Số pixel bão hòa: $N_low = \sum 1[r+c < 0]$; $N_high = \sum 1[r+c > 255]$.

Code skeleton (Google Colab)

```
# Bài 1 - Kiểm soát dtype, overflow/underflow và clipping
```

```
import numpy as np
```

```
def safe_add_uint8(img, c):
    # TODO 1: kiểm tra img có dtype uint8 hay không
    # TODO 2: chuyển sang kiểu có miền biểu diễn rộng hơn (ví dụ int16)
    x = None

    # TODO 3: thực hiện phép cộng
    y = None

    # TODO 4: đếm số pixel sẽ bị clip về 0 và 255
    n_clip_low = None
    n_clip_high = None

    # TODO 5: clip về [0,255] và chuyển lại uint8
    out = None
    return out, n_clip_low, n_clip_high
```

```
# TODO 6: thử với một ảnh nhỏ và so sánh với phép cộng trực tiếp trên uint8
```

Đầu ra bắt buộc / Kết quả mong đợi

- Kết quả đúng phải nằm hoàn toàn trong $[0,255]$.
- Sinh viên phải chỉ ra ít nhất một ví dụ overflow/underflow nếu tính trực tiếp bằng uint8.

Yêu cầu / Nhận xét

1. Vì sao `np.uint8(250) + 30`` có thể không cho 255?
2. Nên chuyển sang `int16``, `float32`` hay `float64`` trong trường hợp nào?

3. Clipping có làm mất thông tin không? Nếu có, thông tin nào bị mất?

Bài 2. Tăng/giảm sáng ảnh

Mục tiêu

Cài đặt brightness transform và phân tích ảnh hưởng của tham số c .

Cơ sở lý thuyết

Biến đổi độ sáng: $s(x,y)=\text{clip}(f(x,y)+c,0,255)$.

Mean: $\mu = (1/N)\sum_i r_i$. Histogram: $h(k)=\sum_i 1[r_i=k]$.

Nếu không clipping, $\mu_s=\mu_r+c$; khi clipping xảy ra, quan hệ này không còn đúng tổng quát.

Code skeleton (Google Colab)

```
# Bài 2 - Tăng/giảm sáng
import cv2
import numpy as np
import matplotlib.pyplot as plt

def adjust_brightness(img, c):
    # TODO 1: chuyển dtype an toàn
    x = None
    # TODO 2: cộng c
    y = None
    # TODO 3: clip [0,255] và đưa về uint8
    out = None
    return out

# TODO 4: chạy với c = -100, -50, 0, 50, 100
results = {}

# TODO 5: hiển thị ảnh trước/sau
# TODO 6: vẽ histogram và in min/max/mean cho từng kết quả
```

Đầu ra bắt buộc / Kết quả mong đợi

- $c>0$: ảnh sáng hơn; $c<0$: ảnh tối hơn.
- Histogram dịch nhưng có thể dồn tại 0 hoặc 255 do clipping.

Yêu cầu / Nhận xét

1. Tại sao histogram không chỉ đơn giản tịnh tiến khi clipping xảy ra?
2. Giá trị mean intensity thay đổi thế nào khi c tăng?
3. Khi nào tăng sáng làm giảm khả năng phân biệt chi tiết vùng sáng?

Bài 3. Pha trộn ảnh (Image Blending)

Mục tiêu

Hiểu weighted sum và ảnh hưởng của α , β , γ .

Cơ sở lý thuyết

Blending: $s(x,y)=\text{clip}(\alpha A(x,y)+\beta B(x,y)+\gamma,0,255)$.

Cross-fade: $s_t = (1-t)A + tB$, $0 \leq t \leq 1$.

Khi $\alpha, \beta \geq 0$, $\alpha + \beta = 1$ và $\gamma = 0$, kết quả chưa lượng tử hóa nằm giữa hai giá trị đầu vào.

Code skeleton (Google Colab)

```
# Bài 3 - Image Blending
import cv2
import numpy as np
import matplotlib.pyplot as plt

def blend_manual(A, B, alpha, beta, gamma=0):
    # TODO 1: kiểm tra A và B cùng kích thước
    # TODO 2: chuyển A, B sang float32
    a = None
    b = None
    # TODO 3: weighted sum
    out = None
    # TODO 4: clip [0,255] và chuyển uint8
    return None

# TODO 5: resize/crop hai ảnh về cùng shape nếu cần
# TODO 6: thử alpha = 0.2, 0.5, 0.8 với beta = 1-alpha
# TODO 7: đối chứng một trường hợp bằng cv2.addWeighted
# TODO 8: tạo 11 frame cross-fade nếu thực hiện challenge
```

Đầu ra bắt buộc / Kết quả mong đợi

- Khi $\alpha + \beta = 1$, output nằm giữa hai ảnh nếu $\gamma = 0$ và không clipping.
- Ảnh kết quả phải thay đổi mượt theo α .

Yêu cầu / Nhận xét

1. Điều gì xảy ra nếu $\alpha + \beta > 1$?
2. γ có vai trò gì?
3. Blending khác phép cộng ảnh thông thường ở điểm nào?

5. Phần B - Các phép biến đổi cường độ

Bài 4. Ảnh âm bản (Negative)

Mục tiêu

Cài đặt phép đảo mức xám và phân tích tính khả nghịch.

Cơ sở lý thuyết

Negative: $s = (L-1) - r$; với $L=256$: $s = 255 - r$.

Tính khả nghịch: $T(T(r)) = r$.

Histogram: $h_s(k) = h_r(255 - k)$.

Code skeleton (Google Colab)

```
# Bài 4 - Negative
import numpy as np
```

```
import matplotlib.pyplot as plt

def negative_uint8(img):
    # TODO 1: kiểm tra miền giá trị/dtype
    # TODO 2: áp dụng  $s = 255 - r$ 
    out = None
    return out

# TODO 3: tính negative một lần và hai lần
neg1 = None
neg2 = None

# TODO 4: kiểm tra tính khả nghịch bằng np.array_equal
is_recovered = None

# TODO 5: hiển thị ảnh gốc, negative và histogram tương ứng
```

Đầu ra bắt buộc / Kết quả mong đợi

- Negative phải đảo mức sáng/tối.
- Áp dụng negative hai lần phải khôi phục ảnh gốc nếu không có phép xử lý khác xen giữa.

Yêu cầu / Nhận xét

1. Negative có làm mất thông tin không?
2. Histogram của ảnh negative liên hệ với histogram ảnh gốc như thế nào?
3. Negative có hữu ích cho ảnh y tế nào và vì sao?

Bài 5. Biến đổi Logarit

Mục tiêu

Cài đặt log transform đúng chuẩn hóa và phân tích vùng tối/sáng.

Cơ sở lý thuyết

Log Transform: $s = c \ln(1+r)$, $r \in [0, 255]$.

$c = 255 / \ln(256)$, nên $T(0) = 0$ và $T(255) = 255$.

Output 8-bit: $\text{round}(\text{clip}(s, 0, 255))$; cần thống nhất quy tắc làm tròn.

Code skeleton (Google Colab)

```
# Bài 5 - Log Transform
import numpy as np
import matplotlib.pyplot as plt

def log_transform(img):
    # TODO 1: chuyển img sang float32
    r = None
    # TODO 2: tính  $c = 255 / \log(256)$ 
    c = None
    # TODO 3:  $s = c * \log(1+r)$ 
    s = None
    # TODO 4: clip, làm tròn theo quy ước và chuyển uint8
    out = None
    return out
```

```
# TODO 5: áp dụng cho ảnh thiếu sáng
# TODO 6: in min/max/mean trước và sau
# TODO 7: hiển thị ảnh và histogram trước/sau
```

Đầu ra bắt buộc / Kết quả mong đợi

- 0 phải ánh xạ về 0 và 255 gần 255.
- Các mức xám thấp tăng tương đối mạnh hơn mức xám cao.

Yêu cầu / Nhận xét

1. Tại sao cần `log(1+r)` thay vì `log(r)`?
2. Cơ số log có ảnh hưởng kết quả nếu c được tính nhất quán không?
3. Khi nào log transform làm mất contrast ở vùng sáng?

Bài 6. Biến đổi Gamma / Power-law

Mục tiêu

Hiểu chuẩn hóa và ảnh hưởng của γ đến độ sáng.

Cơ sở lý thuyết

Chuẩn hóa: $r_n = r/255$, với $r_n \in [0,1]$.

Power-law: $s_n = r_n^\gamma$, $\gamma > 0$.

$s_8 = \text{round}(255 \cdot s_n)$, sau khi clip về $[0,255]$.

$\gamma < 1$: $s_n \geq r_n$; $\gamma = 1$: không đổi; $\gamma > 1$: $s_n \leq r_n$ (với $0 \leq r_n \leq 1$).

Code skeleton (Google Colab)

```
# Bài 6 - Gamma / Power-law
import numpy as np
import matplotlib.pyplot as plt

def gamma_transform(img, gamma):
    # TODO 1: chuẩn hóa uint8 về [0,1]
    r = None
    # TODO 2: s = r ** gamma
    s = None
    # TODO 3: scale về [0,255], clip và chuyển uint8
    out = None
    return out

gammas = [0.3, 0.5, 1.0, 2.0, 3.0]
outputs = {}

# TODO 4: tạo output cho từng gamma
# TODO 5: hiển thị dạng lưới
# TODO 6: vẽ đường truyền  $s=r^\gamma$ 
# TODO 7: in mean intensity để hỗ trợ nhận xét
```

Đầu ra bắt buộc / Kết quả mong đợi

- $\gamma=1$ phải cho ảnh xấp xỉ ảnh gốc.

- $\gamma < 1$ làm sáng; $\gamma > 1$ làm tối, nếu dùng công thức $s = r^\gamma$ trên r đã chuẩn hóa.

Yêu cầu / Nhận xét

1. Tại sao phải chuẩn hóa r về $[0,1]$?
2. Gamma correction có thể đảo ngược không? Nếu có, dùng γ nào?
3. Tại sao cùng γ nhưng hai ảnh có histogram khác nhau có thể cho cảm nhận khác nhau?

6. Phần C - Thresholding và Bit-plane Slicing

Bài 7. Thresholding

Mục tiêu

Cài đặt ngưỡng toàn cục và phân tích foreground/background.

Cơ sở lý thuyết

Threshold: $s(x,y) = 255$ nếu $f(x,y) \geq T$; ngược lại $s(x,y) = 0$.

Foreground count: $N_F = \sum_x \sum_y 1[f(x,y) \geq T]$.

Foreground ratio: $p_F = N_F / (H \cdot W)$.

Code skeleton (Google Colab)

```
# Bài 7 - Thresholding
import numpy as np
import matplotlib.pyplot as plt

def threshold_manual(img, T):
    # TODO 1: tạo ảnh output cùng kích thước
    # TODO 2: pixel >= T -> 255; còn lại -> 0
    out = None
    return out

thresholds = [64, 128, 192]
outputs = {}

# TODO 3: xử lý với từng T
# TODO 4: đếm foreground = số pixel bằng 255
# TODO 5: tính tỷ lệ foreground / tổng số pixel
# TODO 6: hiển thị kết quả và nhận xét ảnh hưởng của T
```

Đầu ra bắt buộc / Kết quả mong đợi

- Output chỉ chứa $\{0, 255\}$.
- Khi T tăng, số pixel foreground không tăng.

Yêu cầu / Nhận xét

1. Thresholding có khả nghịch không?
2. Vì sao threshold quá thấp/quá cao có thể làm mất cấu trúc?
3. Histogram giúp chọn T bằng cách nào?

Bài 8. Lát cắt mặt phẳng bit (Bit-plane Slicing)

Mục tiêu

Tách 8 bit-plane, hiểu đóng góp bit và tái tạo ảnh.

Cơ sở lý thuyết

Tách bit: $B_k(x,y) = (I(x,y) \gg k) \& 1, k=0,\dots,7$.

Biểu diễn: $I(x,y) = \sum_{k=0}^7 B_k(x,y) \cdot 2^k$.

Tái tạo: $\hat{I}_S(x,y) = \sum_{k \in S} B_k(x,y) \cdot 2^k$.

$MAE = (1/N) \sum_i |I_i - \hat{I}_i|$; $MSE = (1/N) \sum_i (I_i - \hat{I}_i)^2$.

$PSNR = 10 \log_{10}(255^2 / MSE)$; $MSE=0$ thì $PSNR = +\infty$.

Code skeleton (Google Colab)

```
# Bài 8 - Bit-plane Slicing
import numpy as np
import matplotlib.pyplot as plt

def extract_bitplane(img, k):
    # TODO 1: kiểm tra 0 <= k <= 7
    # TODO 2: Bk = (img >> k) & 1
    bit = None
    return bit

def reconstruct_from_planes(img, selected_bits):
    # TODO 3: khởi tạo ảnh tái tạo
    rec = None
    # TODO 4: cộng đóng góp Bk * (2**k) cho từng k
    return rec

# TODO 5: tách và hiển thị đủ b0...b7
# TODO 6: tái tạo từ [7], [7,6], [7,6,5], [7,6,5,4]
# TODO 7: tính MAE giữa ảnh gốc và từng ảnh tái tạo
```

Đầu ra bắt buộc / Kết quả mong đợi

- Phải thu được 8 mặt phẳng bit.
- Tái tạo từ b0..b7 phải khớp ảnh gốc.
- Tái tạo chỉ từ bit cao giữ nhiều cấu trúc thị giác hơn bit thấp.

Yêu cầu / Nhận xét

1. Vì sao bit cao thường quan trọng hơn về thị giác?
2. Mất b0 có gây sai số tối đa bao nhiêu cho mỗi pixel?
3. Tái tạo từ b7,b6,b5 có MAE tối đa lý thuyết bao nhiêu?

7. Phần D - Padding

Bài 9. Padding

Mục tiêu

Cài đặt Constant, Replicate và Reflect; phân tích kích thước và giá trị biên.

Cơ sở lý thuyết

Với ảnh $H \times W$ và padding p mỗi phía: $H' = H + 2p$, $W' = W + 2p$.

Constant: $P(y,x) = c$ ngoài miền ảnh; bên trong $P(y+p,x+p) = I(y,x)$.

Replicate: $P(y,x) = I(\text{clip}(y-p, 0, H-1), \text{clip}(x-p, 0, W-1))$.

Reflect: chỉ số ngoài biên được phản xạ về miền hợp lệ; cần nêu rõ quy ước có lặp pixel biên hay không.

Code skeleton (Google Colab)

```
# Bài 9 - Padding
import numpy as np
import cv2
import matplotlib.pyplot as plt

def constant_padding(img, p, value=0):
    # TODO 1: tạo ảnh mới kích thước (H+2p, W+2p)
    out = None
    # TODO 2: chép ảnh gốc vào vùng trung tâm
    return out

def replicate_padding_manual(img, p):
    # TODO 3: tự cài replicate padding
    # Không dùng np.pad / cv2.copyMakeBorder trong phần này
    out = None
    return out

# TODO 4: thực hiện Constant, Replicate và Reflect
# TODO 5: đối chứng với cv2.copyMakeBorder
# TODO 6: in shape trước/sau và hiển thị kết quả
```

Đầu ra bắt buộc / Kết quả mong đợi

- Với ảnh 2×2 , $p=1$, output phải có kích thước 4×4 .
- Constant và Replicate cho biên khác nhau nhưng phần ảnh gốc ở giữa không đổi.

Yêu cầu / Nhận xét

1. Padding có thay đổi dữ liệu gốc bên trong ảnh không?
2. Reflect khác Replicate ở đâu?
3. Vì sao padding quan trọng cho convolution/filtering ở bài sau?

8. Phần E - So sánh và Pipeline tích hợp

Bài 10. Pipeline Point Processing tổng hợp

Mục tiêu

Kết hợp các phép biến đổi có chủ đích và phân tích thứ tự xử lý.

Cơ sở lý thuyết

Pipeline: $I_1 = T_b(I_0)$; $I_2 = T_\gamma(I_1)$; $I_3 = T_n(I_2)$; $I_4 = T_T(I_3)$, tùy cấu hình.

Brightness: $T_b(r) = \text{clip}(r+c, 0, 255)$; Gamma: $T_\gamma(r) = 255(r/255)^\gamma$.

Negative: $T_n(r) = 255 - r$; Threshold: $T_T(r) = 255 \cdot 1[r \geq T]$.

Mean $\mu = (1/N) \sum r_i$; variance $\sigma^2 = (1/N) \sum (r_i - \mu)^2$; $\text{std} = \sqrt{\sigma^2}$.

Các phép biến đổi không nhất thiết giao hoán: $T_A(T_B(I))$ thường khác $T_B(T_A(I))$.

Code skeleton (Google Colab)

```
# Bài 10 - Pipeline Point Processing tổng hợp
import cv2
import numpy as np
import matplotlib.pyplot as plt

def point_pipeline(img, brightness=0, gamma=1.0,
                  use_negative=False, threshold=None):
    # TODO 1: ghi nhận thống kê ảnh đầu vào
    stats_before = None

    # TODO 2: brightness với dtype an toàn
    x = None

    # TODO 3: gamma transform
    x = None

    # TODO 4: negative nếu use_negative=True
    if use_negative:
        x = None

    # TODO 5: threshold nếu threshold khác None
    if threshold is not None:
        x = None

    # TODO 6: thống kê output
    stats_after = None
    return x, stats_before, stats_after

# TODO 7: thử ít nhất 3 cấu hình pipeline
# TODO 8: lưu ảnh trung gian hoặc output
# TODO 9: so sánh min/max/mean/std và histogram
# TODO 10: giải thích vì sao thứ tự các phép biến đổi làm thay đổi kết quả
```

Đầu ra bắt buộc / Kết quả mong đợi

- Mỗi bước phải in min/max/mean để theo dõi biến đổi.
- Sinh viên phải giải thích vì sao chọn thứ tự các phép biến đổi, không chỉ báo ảnh “đẹp hơn”.

Yêu cầu / Nhận xét

1. Thứ tự hai phép point transform có luôn hoán đổi được không?
2. Phép nào làm mất thông tin không thể phục hồi?
3. Nếu threshold làm trước gamma thì gamma còn tác dụng gì trên ảnh nhị phân 0/255?

9. Challenge

- Xây dựng hàm chọn gamma từ mean intensity, xử lý ảnh hằng và ảnh gần đen; giải thích giới hạn của heuristic.
- Tự cài Replicate padding không dùng np.pad hoặc cv2.copyMakeBorder.
- Tìm tập bit-plane nhỏ nhất đạt ngưỡng MAE cho trước; giải thích vì sao kết quả phụ thuộc ảnh.
- Xây dựng pipeline có cấu hình tham số, lưu ảnh trung gian và bảng thống kê; so sánh ít nhất hai thứ tự xử lý.
- Thiết kế bộ kiểm thử tự động cho dtype, clipping, ảnh hằng, ảnh rỗng, shape không khớp và tham số không hợp lệ.

10. Câu hỏi phân tích bắt buộc

- Vì sao phép cộng trên uint8 có thể overflow? Clipping khác wrapping như thế nào?
- Log và Gamma có làm tăng thông tin thực sự của ảnh không? Khi nào biến đổi gây mất thông tin?
- Vì sao histogram giống nhau không đảm bảo ảnh giống nhau?
- Vì sao không nên đánh giá chất lượng cải thiện ảnh chỉ bằng mean hoặc contrast?
- Bit cao và bit thấp có vai trò gì? Có phải bit thấp luôn là nhiễu?
- Padding thay đổi kích thước và thông tin biên như thế nào?
- Vì sao phải thống nhất quy tắc làm tròn khi đổi chứng NumPy và OpenCV?

11. Sản phẩm nộp

Nộp notebook .ipynb chạy từ đầu đến cuối, ảnh đầu vào và kết quả, bảng tham số, biểu đồ/histogram khi phù hợp, câu trả lời phân tích và kết luận. Đặt tên MSSV_HoTen_LAB03.ipynb.